

# Loan Matcher — System Map

How the data, APIs, frontends and login roles connect. Traced from source on 2026-10-01. Companion to `context.md` (the *why*) and `README.md` (the *how to run*).

## 1. What the tool is

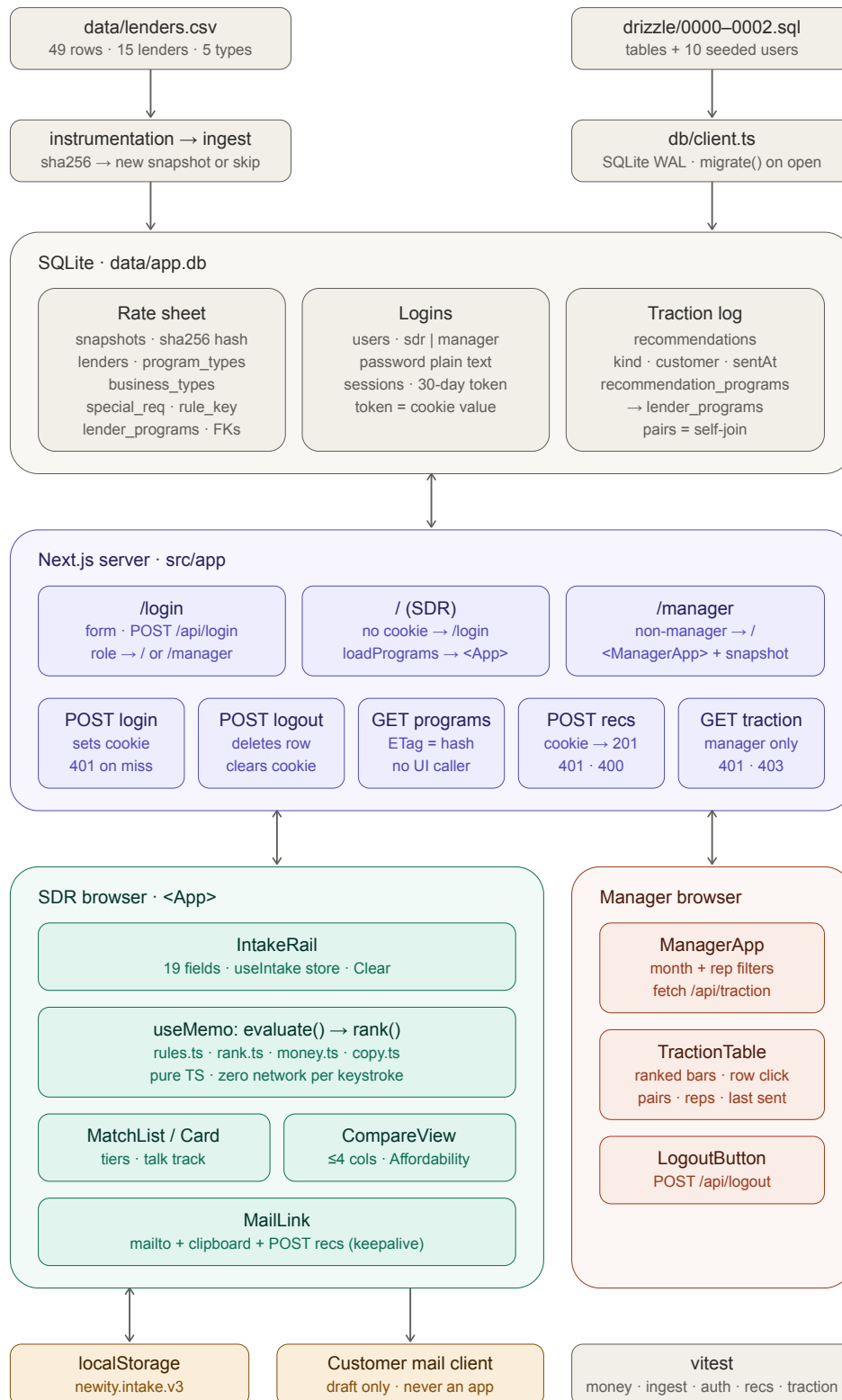
---

A sales-rep screen used live on a phone call. The rep fills a short intake rail; 49 SBA lender programs re-rank on every keystroke as **strong / potential / poor / ineligible**. The rep can open a program (talk track, why-it-fits, watch items, rep-only thresholds), compare up to four side by side, and model affordability with a payment slider. Output to the customer is an email *draft* (`mailto:`), never an application. Each send is logged so sales managers can see what gets recommended and compared.

## 2. The map

---

Figure 1 on the next page shows every moving part and how it is wired. Read top to bottom: sources, boot, SQLite, the Next.js server (pages and APIs), the two browsers by login role, and what leaves the app. Double-headed arrows mean read and write.



■ Data & boot
 ■ Server
 ■ SDR client
 ■ Manager client
 ■ Browser-side externals

Figure 1 — Newity Loan Matcher system map.

### 3. Layer by layer

#### Sources → boot

- `src/instrumentation.ts` runs on server start, calls `ingestCsv` in `src/db/ingest.ts`. The CSV is hashed (sha256). Unseen hash → new `snapshots` row and 49 normalized `lender_programs` rows inside one transaction. Seen hash → "exists", nothing written. Old snapshots are kept.
- An unknown `special_requirements` phrase throws with the phrase in the message, so a human maps it to a `rule_key` in `SPECIAL_KEYS` before the data is trusted.
- `src/db/client.ts` opens `data/app.db` lazily on first use (WAL, foreign keys on) and runs the Drizzle migrations. Migration 0001 creates the login and traction tables and seeds 8 SDRs + 2 managers; 0002 sets every password to `password`.
- Both pages also call ingest inline if a request arrives before boot finished, so the app never serves an empty snapshot.

#### SQLite — three table families

| Family     | Tables                                                                                                                                                                               | Key semantics                                                                                                                                                                                                                                                                                  |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rate sheet | <code>snapshots</code> , <code>lenders</code> ,<br><code>program_types</code> , <code>business_types</code> ,<br><code>special_requirements</code> ,<br><code>lender_programs</code> | Fact rows carry <code>snapshot_id</code> ; the app serves the latest.<br><code>max_debt_ratio</code> NULL = no rule listed.<br><code>business_type_id</code> NULL = All.                                                                                                                       |
| Logins     | <code>users</code> , <code>sessions</code>                                                                                                                                           | <code>users.role = sdr   manager</code> gates every manager path. <code>sessions.id</code> is a random 32-byte token and is literally the cookie value; 30-day expiry.                                                                                                                         |
| Traction   | <code>recommendations</code> ,<br><code>recommendation_programs</code>                                                                                                               | One row per email draft opened ( <code>kind = proposal   comparison</code> , customer name + email, time). Junction pins the exact <code>lender_programs.id</code> . "Compared with" is a self-join on the junction. Aggregation groups by lender + program type so totals survive re-imports. |

#### Next.js server — pages

- Every page and API starts the same way: read the `session` cookie → `userFromToken` in `src/db/auth.ts` → decide.
- `/` (SDR app): no user → redirect `/login`. Loads the full latest snapshot server-side and passes `programs`, `snapshot`, `user` as props to `<App>`. The browser never fetches programs.
- `/manager`: no user → `/login`; SDR → `/`. Renders `<ManagerApp>` with the snapshot summary only; traction data is fetched client-side.
- `/login`: client form with a hard-coded username list (mirrors the seed). POST → cookie → `router.push` by role.

## Next.js server — APIs

| Route                     | Auth            | Does                                                                                                                                                                                |
|---------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST /api/login           | none            | Plain-text compare, inserts session, sets httpOnly / sameSite=lax cookie for 30 days. 401 otherwise.                                                                                |
| POST /api/logout          | cookie optional | Deletes the session row, clears the cookie, 204.                                                                                                                                    |
| GET /api/programs         | none            | Latest snapshot as JSON with ETag = file hash, 304 support, 60 s cache. <b>No UI code calls it.</b>                                                                                 |
| POST /api/recommendations | any user        | Validates {kind, customerName, customerEmail, programIds[]}; proposal = exactly 1 id, comparison $\geq 2$ ; ids must exist. 201 with id, else 400 / 401.                            |
| GET /api/traction         | manager         | ?from&to&rep (to exclusive). One join query, aggregated in JS: totals, ranked programs, top-3 pairs, top-3 senders, last sent, months with sends, rep list. 401 anonymous, 403 SDR. |

## SDR browser

- `src/components/App.tsx` is the hub. `useIntake` (a tiny `useSyncExternalStore` over `localStorage` key `newity.intake.v3`) gives the intake; `programs.map(evaluate).rank(preset)` runs in one `useMemo`. That single `ranked` array feeds the intake rail (for "Unlocks X" hints), the match list, and the compare view.
- Rules, tiering, best-fit score, amortization and all copy are pure functions in `src/lib/` (`rules.ts`, `rank.ts`, `money.ts`, `copy.ts`). Zero network per keystroke.
- Compare set is the rep's `pinned` ids, limit 4, filtered against `ranked`; a program that turns ineligible drops out silently. Selecting a column drives the affordability slider.
- The only server write during a call: a `MailLink` click opens `mailto:`, copies the same draft to the clipboard, and POSTs to `/api/recommendations` with `keepalive`. "Email proposal" logs a proposal; "Send comparison" logs a comparison (one column  $\rightarrow$  proposal).

## Manager browser

- `ManagerApp` holds month + rep filters, fetches `/api/traction` on every change, and redirects to `/login` on a non-OK response. Month defaults to the current month; "All time" is offered.
- `TractionTable` is pure render: ranked bars (compared vs solo width relative to #1), top 8 with "show all", and a row click revealing "Most compared with", "Most recommended by" and "Last sent".

## The loop

SDR send  $\rightarrow$  traction tables  $\rightarrow$  manager view. Snapshot date / counts appear in both top bars from the same `loadPrograms` call. Log out from either role posts to the same endpoint.

## 4. Drift found while tracing

| Where                                              | What                                                                                                                    |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>context.md</code>                            | Says password = username. Migration 0002 and README set every password to <code>password</code> . Doc is stale.         |
| <code>MatchList.tsx</code><br><code>SortRow</code> | Compare helper text says "Top 3 by ..." but <code>COMPARE_LIMIT</code> is 4.                                            |
| <code>/api/programs</code>                         | Dead endpoint: implemented with ETag caching, never called by the UI. Keep as public contract or delete.                |
| <code>login/page.tsx</code>                        | Username dropdown is hand-mirrored from the seed. Adding a user in a migration requires editing this file too.          |
| Design sources                                     | Paper file "Newity Loan Matcher" is the design source of truth; the <code>.docx</code> in the folder is explicitly not. |

## 5. Review order

1. **Data** — row / lender / type counts vs CSV, null handling, all 10 special phrases map, double boot = one snapshot. `ingest.test.ts` .
2. **Rules** — boundary cases per hard rule and per `rule_key` ; tier precedence (hard fail → soft fail → unknown → strong). `money.test.ts` .
3. **APIs** — curl each with no cookie / SDR / manager; shape validation on recommendations. `auth` , `recommendations` , `traction` tests.
4. **Roles** — redirects on `/` and `/manager` ; landing page by role; cookie flags.
5. **Frontends** — type intake → re-rank, open card, compare 3–4, slider, send → row appears in manager view; refresh keeps intake.
6. **Docs** — reconcile the drift table above against `context.md` .

Run first: `npx vitest run` and `npx next build` .